Relatório de uso de Inteligência Artificial

Projeto Laços & Patas — Aplicação de gestão de abrigo de animais

Estudante: [Kyle TEssele Batista (Ana Júlia)] **Turma:** [3 TII]

Data: 21/10/2026 Link site: http://127.0.0.1:5000

Observação sobre o registro: este relatório foi organizado com base nas interações e nos resultados que aparecem na conversa de desenvolvimento. Quando o texto exato de uma pergunta não estava disponível, o pedido foi descrito de forma resumida, sem apresentá-lo como citação literal. Os testes abaixo são apenas os que foram relatados ou observados na conversa; outros testes precisam ser preenchidos depois de realizados.

Interação 1 — Erro de autenticação do PostgreSQL

Ferramenta e data: ChatGPT, durante o desenvolvimento (data exata da conversa anterior não confirmada).

O que pedi: ajuda para entender o erro de autenticação do PostgreSQL que informava falha de senha para o usuário postgres.

O que recebi: orientações para verificar a conexão e considerar um banco PostgreSQL hospedado na nuvem, já que a senha local não estava funcionando.

O que conferi/testei e resultado observado: a tentativa de conexão local apresentava erro de autenticação. Não foi possível confirmar a senha local naquele momento.

O que mudei ou descartei e por quê: deixei de depender da conexão local e passei a configurar o projeto para usar um banco hospedado no Neon.

Comentário crítico / aprendizado: aprendi que um erro de autenticação pode impedir a aplicação de acessar o banco mesmo quando o código está configurado, e que é importante conferir usuário, senha, host e SSL.

Interação 2 — Configuração da conexão com o Neon

Ferramenta e data: ChatGPT, durante a configuração do banco (data exata não confirmada).

O que pedi: orientação para usar a URL de conexão do Neon com o projeto Flask e configurar os dados no pgAdmin.

O que recebi: explicação sobre separar host, banco, usuário, porta e exigência de SSL, além de configurar a variável DATABASE_URL no arquivo .env.

O que conferi/testei e resultado observado: depois de configurar a conexão, os comandos de migração passaram a informar que estavam usando PostgreSQL (PostgresqlImpl). Isso indica que o Flask conseguiu iniciar a comunicação com um banco PostgreSQL.

O que mudei ou descartei e por quê: usei a conexão do banco hospedado em vez de insistir na senha local. A senha de conexão foi compartilhada durante a conversa, então ela deve ser trocada no painel do Neon e não deve ser publicada.

Comentário crítico / aprendizado: aprendi que variáveis de ambiente ajudam a manter configurações fora do código, mas credenciais também precisam ser protegidas e trocadas se forem expostas.

Interação 3 — Conferência do comando de migração atual

Ferramenta e data: ChatGPT, 21/10/2026 (conversa sobre as migrações).

O que pedi: ajuda para interpretar a saída do comando `python -m flask db current`.

O que recebi: explicação de que as mensagens `Context impl PostgresqlImpl` e `Will assume transactional DDL` indicavam que o Alembic estava usando o PostgreSQL e que essas linhas, sozinhas, não eram erros.

O que conferi/testei e resultado observado: executei `python -m flask db current` e vi essas duas mensagens no terminal. A saída mostrada não apresentou uma mensagem de erro, mas também não confirmou por si só que

todas as tabelas estavam criadas.

O que mudei ou descartei e por quê: em vez de concluir que o banco estava pronto apenas por essa saída, continuei para a etapa de aplicar a migração.

Comentário crítico / aprendizado: aprendi a diferenciar mensagens informativas de erro e que preciso verificar o resultado de cada etapa antes de considerar o banco concluído.

Interação 4 — Aplicação da migração inicial

Ferramenta e data: ChatGPT, 21/10/2026.

O que pedi: orientação para continuar depois de conferir o estado das migrações do banco.

O que recebi: instrução para executar `python -m flask db upgrade`, que aplica as migrações pendentes.

O que conferi/testei e resultado observado: executei o comando. O terminal mostrou `Running upgrade -> 93b1e7bea3c2, Modelo inicial do abrigo`, sem exibir erro na saída compartilhada.

O que mudei ou descartei e por quê: não precisei alterar o comando; usei a saída do terminal para seguir à conferência das tabelas no pgAdmin.

Comentário crítico / aprendizado: entendi que os modelos definem a estrutura esperada e que as migrations aplicam essa estrutura ao banco. Não devo criar as tabelas manualmente se o requisito do projeto é usar migrations.

Interação 5 — Verificação das tabelas no pgAdmin

Ferramenta e data: ChatGPT, 21/10/2026.

O que pedi: ajuda para confirmar se a migração criou as tabelas do projeto.

O que recebi: orientação para atualizar a pasta Tables no esquema public do banco e conferir as tabelas e suas Foreign Keys.

O que conferi/testei e resultado observado: a lista mostrada no pgAdmin continha `abrigos`, `animais`, `animal_voluntario`, `fichas_saude`, `voluntarios` e `alembic_version`. A presença das tabelas confirma que a estrutura inicial foi criada, mas ainda é necessário inspecionar as Foreign Keys para verificar todos os relacionamentos.

O que mudei ou descartei e por quê: ainda não descartei nem alterei estruturas. O próximo passo é abrir as Foreign Keys e conferir as ligações entre as tabelas.

Comentário crítico / aprendizado: aprendi que ver as tabelas é uma verificação importante, mas não basta para garantir que os relacionamentos 1:N, N:N e 1:1 estejam corretos.

Uma resposta que não ajudou ou estava errada

O que a IA sugeriu: a primeira tentativa de resolver o acesso ao banco não eliminou o problema da autenticação local; a senha do usuário postgres continuou sendo um impedimento.

Como percebi o problema: o terminal/cliente PostgreSQL retornou uma mensagem de falha de autenticação para o usuário postgres.

Como corrigi ou o que fiz no lugar: optei por configurar a conexão com um banco no Neon e executei as migrações por meio do Flask. A saída posterior mostrou a migração inicial sendo aplicada.

O que aprendi: uma solução precisa ser confirmada com um teste real. Também aprendi a não compartilhar nem deixar pública a senha do banco.

Reflexão final

Durante o desenvolvimento, entendi melhor que o Flask organiza as rotas e a aplicação, enquanto o SQLAlchemy representa as entidades e os relacionamentos do banco. Também aprendi que o Flask-Migrate/Alembic controla as mudanças na estrutura por migrations e que o comando `db upgrade` aplica essas mudanças ao banco configurado. A tabela `animal_voluntario` representa a relação muitos-para-muitos entre animais e voluntários; a

chave estrangeira de `animais` aponta para um abrigo; e a ficha de saúde usa uma chave estrangeira única para manter a relação um-para-um. Na próxima vez, eu verificaria as credenciais antes de começar, protegeria o arquivo `.env` desde o início e testaria cada operação da aplicação, além de conferir as chaves estrangeiras no pgAdmin. Ainda preciso confirmar visualmente as Foreign Keys e testar os cadastros, edições e exclusões antes de afirmar que todas as funcionalidades estão funcionando.